42 SCHOOL PROJECT

Libft

The Complete Beginner's Guide

43	14	3	100%
FUNCTIONS	DAYS PLAN	PARTS	FROM SCRATCH

Open in browser → Ctrl+P → Save as PDF

CHAPTER 01

What is Libft?

Understanding the project before writing a single line of code.

The Simple Explanation

C comes with a standard library full of useful functions like `strlen`, `memcpy`, `printf`. In this project, you recreate many of them yourself — from scratch — so you truly understand how they work at the memory level.

Why it matters

- Every future 42 project will use functions from this library
- You cannot use most standard functions — you must use your own
- It forces you to understand memory, pointers, and strings deeply
- Once done, you have a personal toolbox you carry forever

What you submit

- `libft.h` — header file with all function prototypes and the t_list struct
- `ft_*.c` — one .c file per function (43 total)
- `Makefile` — compiles everything into `libft.a`

The 3 Parts at a Glance

Part	What it contains	Functions	Difficulty
Part 1	Recreate standard libc functions	23 functions	MEDIUM

Part	What it contains	Functions	Difficulty
Part 2	New utility functions not in libc	11 functions	MEDIUM
Part 3	Linked list manipulation	9 functions	HARD

Strict Rules You Cannot Break

✗ **No global variables**

All data must be local to functions or passed as parameters.

✗ **No libtool — use ar**

Create your library with: `ar rcs libft.a *.o`

✗ **No unused files**

Only submit files that are actually needed. Every file must be in Makefile.

✗ **Zero memory leaks**

Every malloc must have a matching free. Valgrind will catch you.

✓ **Helper functions are allowed**

Just declare them as `static` at the top of the .c file.

CHAPTER 02

C Knowledge You Must Master

These are the exact concepts used in this project. Learn these and nothing will surprise you.

1. Pointers — The Most Critical Topic

If you understand only one thing, make it this

A pointer is a variable that stores a **memory address**. It does not store a value — it stores WHERE a value lives in memory.

```
// Regular variable — stores a value
int x = 42;

// Pointer — stores the ADDRESS of x
int *ptr = &x;

// Dereferencing — get the value AT that address
printf("%d\n", *ptr);   // prints 42

// Changing value through pointer
*ptr = 100;
printf("%d\n", x);     // prints 100
```

Double pointers — used in linked list functions

A double pointer `**` is a pointer to a pointer. Used when you need to change what a pointer points to inside a function.

```
// Without double pointer — lst inside won't change outside
void bad_add(t_list *lst, t_list *new) {
    lst = new;  // only changes local copy!
```

```
}

// With double pointer — changes the real pointer
void good_add(t_list **lst, t_list *new) {
    *lst = new;  // changes the actual pointer
}
```

2. Strings — How They Really Work

```
// A string is just a char array ending with '\0'
char *s = "hello";
// Memory: ['h']['e']['l']['l']['o']['\0']
//          s+0   s+1  s+2  s+3  s+4   s+5

// strlen counts UP TO '\0', not including it
ft_strlen("hello") == 5  // not 6!

// Always malloc len + 1 to fit the '\0'
char *copy = malloc(ft_strlen(s) + 1);
```

3. malloc and free — Memory Management

```
// malloc allocates memory on the heap
// returns NULL if allocation fails — always check!
char *s = (char *)malloc(10 * sizeof(char));
if (!s)
    return (NULL);  // handle failure

// use s ...

// free releases it back to the system
free(s);
s = NULL;  // good habit after freeing
```

⚠ **The Golden Rule**

Every single `malloc` must eventually be paired with a `free`. If your function fails halfway

through allocating multiple things, free everything you already allocated before returning NULL.

4. Structs — Used for Linked Lists

```
// Define a struct
typedef struct s_list
{
    void          *content;  // can hold any data type
    struct s_list *next;    // pointer to next node
}   t_list;

// Create a node
t_list *node = malloc(sizeof(t_list));
node->content = "hello";  // use -> for pointer to struct
node->next = NULL;
```

5. Function Pointers — Used in strmapi, lstmap, etc.

```
// Declare: a pointer to a function that takes (void*) and returns void
void (*f)(void *);

// Use it just like a normal function call
f(node->content);

// In a function prototype:
void ft_lstiter(t_list *lst, void (*f)(void *));
//                            ↑ this IS the parameter name
```

6. Casting — Especially for Memory Functions

```
// Always cast to unsigned char in memory/string comparisons
// to avoid sign extension bugs with values > 127
return ((unsigned char)s1[i] - (unsigned char)s2[i]);

// Cast int to long before negating to handle INT_MIN safely
```

```
long nb = (long)n;
if (nb < 0)
    nb = -nb;  // safe: long can hold -INT_MIN
```

CHAPTER 03

Your 14-Day Battle Plan

Exact day-by-day schedule. Follow the order — each day builds on the previous.

Week 1 — Learn C Fundamentals

ℹ Do not skip Week 1. Jumping straight into coding without these foundations is the #1 reason people get stuck and give up.

DAY 1

C Syntax Basics

- → Variables and data types
- → if/else, while loops
- → Writing functions
- → Compile: gcc -Wall -Wextra -Werror
- → Practice: print numbers 1–10

DAY 2

Pointers (Most Important)

- → What is a memory address
- → int *ptr = &x syntax
- → Dereferencing with *ptr
- → Passing pointers to functions
- → Practice: write a swap() function

DAY 3

Arrays and Strings

- → Array = pointer to first element
- → Strings end with \0
- → Loop: while (s[i])
- → Practice: print a string char by char
- → Practice: find length manually

DAY 4

malloc and free

- → Heap vs stack
- → malloc returns NULL on failure
- → Always free what you malloc
- → What a memory leak is
- → Install and try valgrind

DAY 5

Structs and typedef

- → Define a struct
- → Access with . and ->
- → typedef to rename types
- → Practice: create a linked node by hand

DAY 6

Function Pointers

- → void (*f)(void *) syntax
- → Calling: f(argument)
- → Why they exist
- → Practice: pass a function to another function

DAY 7

Setup Your Project

- → Create libft folder
- → Write libft.h with all prototypes
- → Write the Makefile
- → Make sure it compiles (empty lib)
- → Read the full subject PDF again

Week 2 — Build the Library

✓ For every function: read man page → write code → test normal cases → test edge cases → run valgrind. Only then move on.

DAY 8

Memory Functions

- → ft_memset
- → ft_bzero → uses memset
- → ft_memcpy
- → ft_memmove → uses memcpy
- → ft_memchr
- → ft_memcmp

DAY 9

Character Functions

- → ft_isalpha
- → ft_isdigit
- → ft_isalnum → uses both above
- → ft_isascii
- → ft_isprint
- → ft_toupper
- → ft_tolower

DAY 10

String Functions

- → ft_strlen ← foundation
- → ft_strchr
- → ft_strrchr
- → ft_strncmp
- → ft_strnstr → uses memcmp
- → ft_strlcpy
- → ft_strlcat
- → ft_atoi

DAY 11

Allocation Functions

- → ft_strdup → strlen + memcpy
- → ft_calloc → malloc + bzero
- → ft_substr ← key function
- → ft_strjoin

DAY 12

Harder Functions

- → ft_strtrim → uses substr
- → ft_itoa (careful: INT_MIN!)
- → ft_split ← hardest one

DAY 13

Output + Iteration

- → ft_putchar_fd
- → ft_putstr_fd
- → ft_putendl_fd
- → ft_putnbr_fd → uses putchar_fd
- → ft_strmapi
- → ft_striteri

DAY 14

Linked List Functions

- → ft_lstnew
- → ft_lstadd_front
- → ft_lstsize
- → ft_lstlast
- → ft_lstadd_back → uses lstlast
- → ft_lstdelone
- → ft_lstclear → uses lstdelone
- → ft_lstiter
- → ft_lstmap → uses lstnew + lstadd_back + lstclear

BUFFER DAY

Test Everything

- → Run francinette tester
- → Fix any failing tests
- → Run valgrind on all functions
- → Check Makefile rules work
- → Read your code out loud

CHAPTER 04

All 43 Functions — Quick Reference

What each function does, what to watch out for, and which difficulty to expect.

Part 1 — Libc Functions

Function	What it does	Watch out for	Level
`ft_isalpha`	Returns 1 if c is a letter	Must return 1 or 0, not nonzero	EASY
`ft_isdigit`	Returns 1 if c is 0–9	Same as above	EASY
`ft_isalnum`	Returns 1 if letter or digit	Reuse isalpha + isdigit	EASY
`ft_isascii`	Returns 1 if c is 0–127	Includes 0	EASY
`ft_isprint`	Returns 1 if printable (32–126)	Space (32) is printable	EASY
`ft_toupper`	Converts lowercase to uppercase	Return unchanged if not lowercase	EASY
`ft_tolower`	Converts uppercase to lowercase	Return unchanged if not uppercase	EASY
`ft_strlen`	Count chars until \0	Does NOT count the \0	EASY
`ft_memset`	Fill memory with a byte value	Cast to unsigned char	EASY
`ft_bzero`	Fill memory with zeros	Just call memset(s, 0, n)	EASY
`ft_memcpy`	Copy n bytes from src to dst	Undefined if they overlap	EASY

Function	What it does	Watch out for	Level
`ft_memmove`	Copy n bytes, handles overlap	Copy backwards if dst > src	MEDIUM
`ft_strlcpy`	Copy string safely into dst	Always null-terminates, returns src length	MEDIUM
`ft_strlcat`	Append string safely	Returns total desired length, not actual	MEDIUM
`ft_strchr`	Find first occurrence of char	\0 is a valid search target	EASY
`ft_strrchr`	Find last occurrence of char	\0 should return pointer to end	EASY
`ft_strncmp`	Compare first n chars	Cast to unsigned char before subtracting	MEDIUM
`ft_memchr`	Find byte in memory block	Use unsigned char for comparison	EASY
`ft_memcmp`	Compare two memory blocks	Cast to unsigned char	EASY
`ft_strnstr`	Find needle in haystack (max len)	Empty needle returns haystack	MEDIUM
`ft_atoi`	String to integer	Skip whitespace, handle +/- sign	MEDIUM
`ft_calloc`	Allocate zeroed memory	Return unique ptr if nmemb or size is 0	MEDIUM
`ft_strdup`	Duplicate a string	Must malloc, returns NULL on fail	EASY

Part 2 — Additional Functions

Function	What it does	Watch out for	Level
`ft_substr`	Extract substring from string	start >= strlen → return ""	MEDIUM
`ft_strjoin`	Concatenate two strings into new one	Malloc len1 + len2 + 1	EASY
`ft_strtrim`	Remove chars from both ends	set is a SET of chars, not a string to match	MEDIUM
`ft_split`	Split string by delimiter into array	Array must end with NULL pointer	HARD
`ft_itoa`	Integer to string	INT_MIN: cast to long before negating	MEDIUM
`ft_strmapi`	Apply function to each char, return new string	Malloc new string, pass index + char	MEDIUM
`ft_striteri`	Apply function to each char in place	Pass address of char — modifies original	EASY
`ft_putchar_fd`	Write one char to file descriptor	Use write(), not printf()	EASY
`ft_putstr_fd`	Write string to file descriptor	Parameter is char*, not char const*	EASY
`ft_putendl_fd`	Write string + newline to fd	Two write() calls	EASY
`ft_putnbr_fd`	Write integer to fd	Use long, handle INT_MIN	MEDIUM

Part 3 — Linked List Functions

Function	What it does	Watch out for	Level
`ft_lstnew`	Create a new node	content is stored directly, not copied	EASY

Function	What it does	Watch out for	Level
`ft_lstadd_front`	Add node at beginning	new->next = *lst, then *lst = new	EASY
`ft_lstsize`	Count number of nodes	Returns int, not size_t	EASY
`ft_lstlast`	Return last node	NULL input → return NULL	EASY
`ft_lstadd_back`	Add node at end	Reuse ft_lstlast	EASY
`ft_lstdelone`	Free one node using del function	Do NOT free next — just this node	MEDIUM
`ft_lstclear`	Free all nodes from point onward	Set *lst = NULL at the end	MEDIUM
`ft_lstiter`	Apply function to content of each node	Pass content, not the node itself	EASY
`ft_lstmap`	Create new list by mapping function over each node	On malloc failure, clear new list with del	HARD

CHAPTER 05

Function Reuse Map

Never rewrite what you already wrote. Here is every place you can reuse a function you built.

Complete Dependency Tree

```
ft_memset └── ft_bzero └── ft_calloc ft_memcpy └── ft_memmove (when no overlap)
└── ft_strdup └── ft_substr └── ft_strjoin ft_memcmp └── ft_strnstr ft_strlen ├──
ft_strlcpy ├── ft_strlcat ├── ft_strdup ├── ft_strjoin ├── ft_strnstr ├──
ft_putstr_fd ├── ft_putendl_fd └── ft_substr ├── ft_strtrim └── ft_split
ft_strdup └── ft_substr (empty string edge case) ft_putchar_fd └── ft_putnbr_fd
ft_lstlast └── ft_lstadd_back └── ft_lstmap ft_lstnew └── ft_lstmap ft_lstdelone
└── ft_lstclear └── ft_lstmap (on failure cleanup)
```

Why ft_substr is the Most Important Function

ft_substr is used by 3 other functions

Get ft_substr perfect before building ft_strtrim, ft_split, and (indirectly) anything that needs a portion of a string. A bug in ft_substr will break all three.

```
// ft_strtrim just finds the boundaries, then calls:
return (ft_substr(s1, start, end - start));

// ft_split builds each word with:
arr[wi] = ft_substr(s, start, word_length);

// ft_substr itself handles empty string edge case with:
if (start >= slen)
    return (ft_strdup(""));
```

CHAPTER 06

The Hard Functions — Explained

The functions that trip up most beginners, with clear step-by-step thinking.

ft_memmove — The Overlap Problem

The issue: if `dst` and `src` overlap in memory, copying forward will corrupt data you haven't read yet. Solution: copy backwards when `dst` is inside the source region.

```
// Overlap case — copy forwards would corrupt:
// src: [1][2][3][4][5]
// dst:    [?][?][?][?][?]   (dst = src + 1)
// Copying forward: dst[0]=src[0]=1, then dst[1]=src[1]
// but src[1] is now dst[0]=1 → CORRUPTED

// Solution: when dst > src and inside src, copy backwards
if (d > s && d < s + len)
{
    d += len;
    s += len;
    while (len--)
        *--d = *--s;  // copy from end to start
}
else
    ft_memcpy(dst, src, len);  // safe to use memcpy
```

ft_atoi — String to Integer

```
// Input: "  -42abc"  → output: -42
// Steps:

// Step 1: skip whitespace
```

```
while (*str == ' ' || (*str >= '\t' && *str <= '\r'))
    str++;

// Step 2: read sign
if (*str == '-' || *str == '+')
{
    if (*str == '-') sign = -1;
    str++;
}

// Step 3: read digits until non-digit
while (*str >= '0' && *str <= '9')
{
    result = result * 10 + (*str - '0');
    str++;
}

return (sign * result);
```

ft_itoa — The INT_MIN Trap

✗ **INT_MIN = -2147483648**

Doing `n = -n` when n is INT_MIN causes overflow because 2147483648 doesn't fit in int. Cast to long first.

```
long nb = (long)n;    // safe cast before any negation
if (nb < 0)
    nb = -nb;          // now safe: long can hold 2147483648
```

ft_split — Think Before You Code

Draw this on paper first

Input: `"hello::world::foo"` with delimiter `':'`

Output: `["hello", "world", "foo", NULL]`

```
// Step 1: count words (needed to malloc the right size)
// "hello::world::foo" → 3 words

// Step 2: malloc array of (words + 1) pointers
// +1 for the NULL terminator at the end
arr = malloc((words + 1) * sizeof(char *));

// Step 3: for each word, use ft_substr to extract it
while (wi < words)
{
    while (s[i] == c) i++;      // skip delimiters
    start = i;
    while (s[i] && s[i] != c) i++;  // find end of word
    arr[wi] = ft_substr(s, start, i - start);
    if (!arr[wi]) { free_all(arr, wi); return (NULL); }
    wi++;
}
arr[wi] = NULL;  // terminate with NULL
```

ft_lstmap — The Most Complex Function

```
// Goal: create a NEW list by applying f() to each node's content
// If any malloc fails, clear the new list and return NULL

t_list *new_list = NULL;

while (lst)
{
    // Create new node with f applied to content
    t_list *node = ft_lstnew(f(lst->content));

    if (!node)
    {
        // Malloc failed — clean up everything built so far
        ft_lstclear(&new_list, del);
        return (NULL);
    }

    ft_lstadd_back(&new_list, node);  // add to new list
```

```
        lst = lst->next;
    }
    return (new_list);
```

CHAPTER 07

Testing and Debugging

How to know your code actually works — not just that it compiles.

Your Daily Checklist for Every Function

- [ ] Read the man page before writing anything
- [ ] Write what the function does in plain English (on paper)
- [ ] Write the code
- [ ] Test: normal/expected input
- [ ] Test: empty string ""
- [ ] Test: NULL input
- [ ] Test: size 0 or length 0
- [ ] Test: boundary values (start past end, etc.)
- [ ] Compare output with real libc function
- [ ] Run valgrind — zero leaks, zero errors

How to Write a Test File

```
// test.c — make one of these for each function
```

```c
#include <stdio.h>
#include <string.h>    // for the real function
#include "libft.h"

int main(void)
{
    // Test ft_strlen
    printf("strlen test:\n");
    printf("yours: %zu | real: %zu\n", ft_strlen("hello"), strlen("hello"));
    printf("yours: %zu | real: %zu\n", ft_strlen(""), strlen(""));

    // Test ft_substr
    printf("\nsubstr test:\n");
    printf("%s\n", ft_substr("hello world", 6, 5));  // "world"
    printf("%s\n", ft_substr("hello", 10, 3));        // ""

    return (0);
}
```

```bash
# Compile and run
gcc -Wall -Wextra -Werror -o test test.c ft_strlen.c ft_substr.c ft_strdup.c ft_memc
./test
```

Using Valgrind — Zero Tolerance for Leaks

```bash
# Basic leak check
valgrind --leak-check=full ./test

# Full output including where leak happened
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./test

# What you want to see:
# LEAK SUMMARY: definitely lost: 0 bytes in 0 blocks
# ERROR SUMMARY: 0 errors from 0 contexts
```

⚠ **Common leak pattern in ft_split**

If malloc fails on arr[3], you must free arr[0], arr[1], arr[2], and arr before returning NULL. A helper

function `free_all(arr, count)` keeps this clean.

Edge Cases That Fail Most Submissions

Function	Tricky input	Expected result
`ft_substr`	start >= strlen(s)	Return "" (empty string, not NULL)
`ft_split`	"::::" (only delimiters)	[NULL] — empty array
`ft_split`	"" empty string	[NULL] — empty array
`ft_atoi`	" +42"	42
`ft_atoi`	"42abc"	42 (stop at non-digit)
`ft_itoa`	-2147483648	"-2147483648" (use long!)
`ft_strchr`	searching for '\0'	Pointer to end of string
`ft_strtrim`	set = "" (empty set)	Copy of s1 unchanged
`ft_lstdelone`	lst = NULL	Do nothing, no crash
`ft_calloc`	nmemb = 0 or size = 0	Return unique pointer (not NULL)

CHAPTER 08

Top Tips to Finish Without AI

The habits and mindset that separate students who finish from those who don't.

---

1 **Read the man page before writing anything**

Type `man strlen` before writing ft_strlen. The man page is your specification. It tells you exactly what the function must do, what it returns, and what edge cases exist.

2 **Write it in plain English first**

On paper, write what the function does in simple sentences before writing code. "Get the length of the string. Allocate length + 1 bytes. Copy bytes. Add null terminator." That's your ft_strdup.

3 **Draw pointers and linked lists on paper**

For anything with pointers or linked lists, literally draw boxes and arrows. 10 minutes drawing saves 2 hours debugging. Your brain cannot reliably visualize pointer chains without help.

4 **Test immediately after each function**

Don't write 5 functions then test. Write one, test it fully with normal and edge cases, run valgrind, then move on. Bugs found immediately take 5 minutes to fix. Bugs found later take hours.

5 **Always check malloc return value**

Every single malloc can return NULL. If you don't check, your program will segfault with impossible-to-debug errors. Check every time: `if (!ptr) return (NULL);`

6 **Run valgrind every day — zero tolerance**

One leaked malloc will fail your evaluation. Don't let leaks pile up thinking you'll fix them later. Fix every leak the moment valgrind reports it.

7 **Ask ft_strlen "what does this reuse?"**

Before writing any function, scan what you've already built. ft_strtrim is just ft_substr with some index

math. ft_bzero is just ft_memset with 0. Recognizing this saves time and reduces bugs.

8 **If stuck for 30 minutes, take a walk**

Your brain solves problems during rest. The answer that evades you after 30 minutes of staring often becomes obvious after a 10-minute walk. This is not procrastination — it's how thinking works.

9 **Ask a peer, not a search engine**

Explaining your problem to another person forces you to articulate it clearly. Often the act of explaining reveals the bug. Peers also give context-aware help that generic resources can't.

10 **Understand before moving on**

If your function passes the tests but you don't understand why it works, you haven't learned it yet. During peer evaluation you'll be asked to explain every line. Write code you can explain.

Good Practice vs Bad Practice

✓ Good

- Stuck on ft_split → draw it on paper
- Read man page → understand → write
- Test one function fully before next
- Ask a peer to explain their approach
- Valgrind after every function
- Write code you can explain line by line

✗ Bad

- Copy code without understanding it
- Skip testing to "go faster"
- Ignore valgrind warnings
- Write 10 functions then test all at once
- Move on when confused hoping it's fine
- Memorize code without understanding why

CHAPTER 09

The Makefile Explained

The Makefile is required and must have specific rules.

```
NAME    = libft.a
CC      = cc
CFLAGS  = -Wall -Wextra -Werror

SRCS    = ft_strlen.c ft_memset.c ... # all 43 .c files
OBJS    = $(SRCS:.c=.o)               # auto-convert .c to .o

all: $(NAME)                          # default rule

$(NAME): $(OBJS)
    ar rcs $(NAME) $(OBJS)            # create the .a library

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@    # compile each .c to .o

clean:
    rm -f $(OBJS)                     # remove object files

fclean: clean
    rm -f $(NAME)                     # remove library too

re: fclean all                        # full rebuild

.PHONY: all clean fclean re
```

Must use ar, not libtool

`ar rcs libft.a *.o` — r=insert, c=create, s=add index. The subject explicitly forbids libtool.

Makefile must not relink

Running `make` twice should not recompile anything the second time. The `%.o: %.c` rule

handles this automatically since make checks file timestamps.

FINAL WORD

Remember This

The mindset that gets you through any 42 project.

Build the foundation first

ft_strlen, ft_memset, ft_memcpy. Everything else builds on these. Get them solid.

Never rewrite what you have

Before coding, scan your existing functions. The answer is almost always already there.

Test before moving on

One function done properly beats three functions done sloppily. Quality over speed.

The one question to ask after every malloc

"Where does this get freed?"

If you cannot answer that question, you have a memory leak waiting to happen. Find the answer before moving on.

✓ **You will get stuck. That is normal.**

Stuck means you are at the edge of what you currently understand. That edge is exactly where learning happens. Push through it.